ESWC 2027

Frozen Research Scope & Experimental Protocol

Reusable Semantic Capability Knowledge for Heterogeneous Web API Integration and LLM-Agent Repair

Scope freeze: 17 October 2026 | Status: pilot protocol, not evidence of research novelty or results

Purpose. Consolidate the original Semantic Web Environments motivation, the research-plan discussion, the simulated critical review, Experimental Protocol v2, and the final methodological decisions into one implementation-ready reference. This document preserves unresolved operational choices explicitly rather than presenting them as settled facts.

Frozen central decision

> **E1 - cross-provider portability cost subject to correctness - is primary. E2 - LLM versus classical planning/repair with identical formal goals - is secondary. E3 - ambiguous natural-language interpretation - is exploratory and may be deferred. The project tests semantic knowledge management and reuse, not RDF serialization superiority.**

Provisional paper title: Reusing Semantic Capability Knowledge for LLM-Agent Interoperability: A Controlled Study of Heterogeneous Web API Integration.

Evidence standard: Null or negative findings are legitimate; they cannot be recast as positive evidence. A pilot is not itself sufficient for general claims across providers.

1. Scientific positioning and claim boundaries

The starting work describes Agent & Artifact environments, Web of Things descriptions, and semantic Usage manuals with operations, preconditions, and postconditions. Semantic service discovery, mediation, substitution, and goal-oriented composition have extensive prior art. Likewise, LLM planning, symbolic verification, and feedback-driven repair are not claimed as new.

Potential contribution: A controlled empirical account of the total cost, correctness, and limits of reusing semantic capability knowledge across independently authored Web API providers, plus a secondary test of what LLM-assisted planning contributes beyond classical planning.

Excluded claims: (i) RDF is intrinsically more reliable than JSON; (ii) ontology mediation is newly invented; (iii) an LLM is necessarily superior to classical planning; (iv) an HTTP success response proves a goal; (v) scripted API tests establish sustained real-world use.

Research questions

Question	Frozen wording / role
RQ1 - Primary	To what extent does reusable ontology-mediated capability knowledge reduce total and incremental effort for integrating independently developed Web APIs while preserving correct execution?
RQ2 - Secondary	Given identical formal goals and integrated services, when does LLM-assisted planning and repair improve over classical semantic planning?
RQ3 - Exploratory	How accurately does an LLM formalize ambiguous natural-language requests and request clarification when needed? May be deferred.

2. Hybrid architecture and independence rules

Hybrid model: Provider A, B, and held-out C each have independently maintained capability manuals. A shared registry exposes common ontology concepts, mapping assertions, and discovery endpoints. Generic validation, planning interfaces, and execution adapters use these descriptions at runtime.

- Provider manual: locally named operations; input/output schemas; critical preconditions; modeled transitions and effects; API bindings; evidence/observation requirements; documentation provenance.
- Shared registry: provider/manual discovery, shared vocabulary, versioned reusable mappings, and provenance of mappings. It must not encode task-specific plans or a direct A-to-B substitution table.
- LLM: candidate plan proposals, interpretation of feedback, and optional goal formalization. The LLM is not the authority for action validity.
- Symbolic layer: capability mediation, precondition evaluation, candidate filtering, transition reasoning, and structured violation evidence.
- Executor: HTTP invocation, explicit observation/read-after-write, and audit trace. Declared and observed effects are logged separately.

Authoring independence: Authors may see the common vocabulary and agreed manual schema, but not held-out tasks, oracle answers, or target mappings. Record who authored each manual, examples available, and all central mapping intervention.

3. Formal contract and operational semantics

An operation is a = (I, P, T, B, O), where I is its input schema/constraints; P its critical state preconditions; T(s,s′) its modeled transition relation; B its concrete API binding; and O its observation/evidence specification. A goal G consists of required state predicates and mandatory invariants.

Applicability, progress, achievement and verification

- Applicability: Applicable(a,s,x) iff input x satisfies I and every critical precondition in P is established true from admissible state evidence.
- Three-valued evidence: preconditions are true, false, or unknown. Unknown critical facts require observation, clarification, or rejection; missing RDF facts are never treated as true.
- Progress: at least one modeled successor can advance an outstanding subgoal without violating invariants. Progress does not imply complete goal achievement.
- Achievement: observed post-execution state s′ satisfies all goal predicates and invariants.
- Verification: O specifies fresh and sufficient evidence to support the achievement claim. An HTTP 2xx alone is insufficient.

Substitution and repair

A replacement b is considered only if it is applicable in the current state, its input/output transformations are explicitly available, and its modeled transitions support the outstanding goal. Distinguish existential feasibility (some modeled outcome succeeds) from conservative preservation (all relevant successful modeled outcomes meet the obligation). Neither guarantees real-world API behavior.

Repair feedback records: action, constraint identifier, verdict (false/unknown), observed facts, evidence source, and proposed next step. Repaired plans are checked again before execution.

4. Experimental separation and controlled comparisons

Experiment	Inputs held fixed	Primary question / output
E1 - Portability	Formal goals; held-out providers; frozen generic machinery	Total and marginal integration effort, subject to independently checked correctness.
E2 - Planning	Same formal goals, available actions, observations, budgets and validation feedback	LLM versus state-updated classical planning: success, repair, invalid actions, latency/cost.
E3 - Interpretation (optional)	Natural-language goals; fixed service universe; independent oracle	Goal formalization, acceptable interpretations, prohibited assumptions, appropriate clarification.

E1 infrastructure conditions

A - API-local	B - JSON + shared semantics	C - RDF + shared semantics
Provider-local contracts, mappings and validators. Engineering baseline; not information-equivalent to shared-knowledge systems.	JSON contracts carrying ontology identifiers, reusable mappings, shared semantics and equivalent validation/reasoning capabilities.	Independent RDF manuals, ontology-mediated registry, reusable mappings and validators.

Parity requirement: B and C receive the same capability meanings, constraints, effects, mappings, transformations, observations and validation power. Audit functional parity before any performance comparison. A-to-(B/C) isolates shared semantic infrastructure; B-to-C isolates RDF-native implementation choices, not semantic knowledge availability.

E2 classical/LLM fairness

Use identical formal goals and observations. Allow the classical planner to replan after state changes; use the same execution budget and validation feedback. Record invalid proposals separately from invalid executed actions. E3, if undertaken, uses an independent oracle allowing multiple acceptable interpretations and mandatory clarification cases.

5. Providers, held-out integration, and leakage prevention

Pilot domain: three calendar-like providers with independently authored documentation and manuals. A and B are visible for initial integration; C is sealed until the generic system is frozen. Weather and public-holiday APIs may supply context but should not dominate E1.

Provider	Designed challenge
A - initial	Conventional create/update/read operations; baseline integration.
B - initial	Different names, parameters, output formats and capability vocabulary; positive cross-provider mappings.
C - held out	Unseen interface with positive and negative matches: misleadingly similar operation names, non-equivalent effects, restricted permissions, missing evidence or unsupported transitions.

Procedure: (1) independently author A/B manuals; (2) independently prepare and seal C plus evaluation tasks/oracle; (3) freeze generic discovery, validator, planner interfaces, prompts and registry logic; (4) reveal C; (5) permit only C-specific manuals, bindings and mappings; (6) log all changes and failures; (7) test both safe rejection and verified execution.

Not permitted: modifying frozen generic code without recording protocol deviation; provider-to-provider substitution tables; task-specific repair scripts; post-hoc manual changes informed by oracle answers without disclosure.

6. Complete cost accounting and break-even analysis

C_total(n) = C_setup + sum(i=1..n) C_onboard(i) + C_maintenance(n). Cost is measured from a declared starting point, not only after the shared ontology exists.

Cost category	Included work
Setup	Ontology/vocabulary design; common mapping curation; shared registry; validator and reusable mediation implementation; adaptation of tutorial agent; initial tests.
Onboarding per provider	Reading documentation; authoring manuals/contracts; new mappings; input/output adapters; provider-specific rules; tests; failed attempts; debugging.
Maintenance	API revisions; repair of mappings/manuals; validator updates; regression tests; documentation updates.

Measurement record: elapsed person-minutes; integrator identity and experience; edited files/artifacts; mapping assertions added/reused; rules; lines changed; tests; failures; debugging; tool/LLM assistance; correctness at completion. If only one integrator is available, disclose the limitation.

Cold start, incremental reuse, and amortization

- Cold start: all study-specific infrastructure and initial provider work are charged to the condition.
- Incremental: onboarding held-out C after A/B are integrated; distinguish reused knowledge from newly authored alignments.
- Break-even: plot observed cumulative costs for n = 1, 2, 3. If no crossing occurs, report no observed break-even. Any extrapolated crossing is explicitly a model-based sensitivity estimate with assumptions about future provider mix, mapping reuse, labor cost and maintenance.

Interpretation: A faster marginal integration does not prove lower cumulative cost. The study must report both and may find that semantic infrastructure remains more expensive across the observed sample.

7. Correctness oracle and negative controls

Before running systems, independently specify per-task required end-state predicates, forbidden side effects, acceptable alternatives, and observation requirements. Use fixture-controlled API responses and read-after-write or state queries for writable services.

- Count false-positive substitutions, false-negative rejections, valid call execution, verified end-state achievement, and unknown-state handling separately.
- Include deceptive labels, different operations with equivalent effects, operations with similar labels but incompatible effects, missing permissions, missing evidence and service failures.
- A condition is counted as successfully integrated only if required tasks are achieved or safely rejected as appropriate. Unsafe execution is a failure even when HTTP returns success.

8. Sampling, statistical analysis, and limits

Begin with 20 diagnostic pilot cases covering valid actions, multi-step goals, failures, semantic mismatch, and invalid substitutions. These cases validate the apparatus; they do not support strong generalization. Extend to more held-out providers, independent manual authors and integrators if feasible.

Provider is the primary unit of generalization for onboarding cost. Repeated LLM runs on the same provider/task are not independent provider samples. Publish raw task-level results, paired differences, uncertainty intervals, failure taxonomy and protocol deviations. Predefine retry limits, timeout, repair iteration budget and human-intervention rules before confirmatory runs.

Evidence that would support different conclusions

Observation	Defensible conclusion
Shared semantics reduces marginal effort without correctness loss	Evidence of reusable knowledge benefits in the evaluated provider setting; report setup and maintenance costs.
B and C perform similarly	Shared ontology/mapping reuse matters more than RDF-native representation for these tasks.
No observed break-even	Do not claim economic superiority; report observed costs and conditional model projections.
Classical planning matches LLM on formal goals	LLM planning benefit not established for fully specified tasks; any NL interpretation result belongs to E3.
Semantic mappings require extensive curation	Characterize curation economics; do not call interoperability automatic.
Negative or null pilot	Valid feasibility finding; reconsider full-paper claims and scope.

9. Go / no-go decision procedure

Gate 1 - Readiness (required before confirmatory E1)

Executable contract semantics; independent manuals; sealed held-out C and tasks; verified B/C parity; frozen generic machinery; complete cost logs; negative substitution cases; independent observable oracle. If any is missing, continue engineering but do not label runs confirmatory.

Gate 2 - Evidence (direction-neutral)

Report correctness and cumulative/marginal costs regardless of sign. Accept that reuse may improve, worsen, or leave outcomes indistinguishable. A negative result does not trigger selective metric changes or unreported redesign.

Gate 3 - Research Track decision

Proceed only if the final study supports a differentiated, generalizable empirical finding, with full-text comparison to classical semantic service mediation and recent neuro-symbolic agent work. If evidence remains a small scripted prototype, prefer demo/resource positioning over inflated research claims.

10. Implementation boundary and staged plan

Reuse target: adapt the referenced Agentic AI tutorial for LLM invocation, tool calling and agent execution loop, subject to an implementation/license audit. No verified mapping of tutorial classes/modules is claimed here. Do not rebuild a full multi-agent framework.

Minimal vertical slice: one independent manual, one formal goal, one executable API contract, one applicability decision, one API call, one observed postcondition, and one evidence-backed explanation. Then integrate provider B, freeze generic code, and test held-out C.

Suggested stack, not frozen dependencies: Python agent runtime (tutorial adaptation); RDFLib/SPARQL; SHACL or narrow procedural checks for constraints; HTTPX/FastAPI for controlled services; JSON logs plus optional RDF provenance. Tool choices may change without altering the experimental protocol, provided costs are logged.

Phase	Priority	Deliverable
Before coding	Mandatory	Finalize contract vocabulary, provider specifications, sealed oracle, mapping ownership and cost-log template.
Vertical slice	Mandatory	One formally checked, executed and observed action; inspect execution trace.
E1 pilot	Mandatory	A/B integration, frozen system, held-out C onboarding under A/B/C conditions.
E2 pilot	Secondary	Classical vs LLM planning and repair with identical formal goals.
E3 pilot	Optional	Ambiguous NL goals and independent interpretation oracle.
Paper decision	Evidence based	Research vs In-Use vs demo/resource positioning based on results and deployment maturity.

11. Known unresolved choices (must be recorded, not silently assumed)

- Exact contract ontology IRIs, SHACL shapes, and supported transition-language fragment.
- Concrete API implementations and how provider authors/integrators are assigned independently.
- Frozen retry budget, timeouts, task distribution, mapping policy, and numerical success thresholds.
- Whether sufficient providers/integrators can be recruited for a generalizable full-paper evaluation.
- Whether any real operational deployment exists; public API invocation alone does not establish In-Use maturity.

12. Provenance and closest-work checklist

Project sources: original Semantic Web Environments report (provided 2003.02054v1.pdf); initial seven-page research plan; ESWC 2027 simulated critical review; Experimental Protocol v2 (17 Oct 2026); final scope-freeze decisions in this conversation.

Literature requiring full-text feature comparison before novelty claims: OWL-S (Martin et al.); WSMO (Roman et al.); semantic service mediation (Burstein & McDermott; Dietze et al.); WSMO-Lite/hRESTS; ontology-based service adaptation (Staikopoulos et al.); AUTOBUS (Pang & Sayama, 2026); ontology-to-tools work (2026); LLM-to-symbolic-goal and neuro-symbolic planning studies. These are a review checklist, not a claim that every cited work has been independently verified in full text.

Submission constraint: ESWC 2027 main-track abstract and paper deadlines discussed in project materials are 10 and 17 December 2026 respectively; recheck the official track call before submission.

> **Freeze statement: RQ1/E1 primary; RQ2/E2 secondary; RQ3/E3 optional. Correctness-constrained integration economics, independent held-out providers, B/C information parity, observed-versus-projected break-even, and direction-neutral go/no-go decisions are fixed. Operational parameters listed above remain to be specified before confirmatory data collection.**